

编译原理实验 PR001

14 小组成员：谢翊凡、曹建文、闫涛

一、实验代码简介：

1, 语法规则部分代码：

```
grammar > CACT.g4
1  grammar CACT;
2
3  @header {
4      #include <vector>
5  }
6
7  // 编译单元
8  compUnit
9      : (decl | funcDef)+ EOF
10     ;
11
12 // 声明
13 decl
14     : constDecl
15     | varDecl
16     ;
17
18 // 常量声明
19 constDecl
20     : 'const' bType constDef (',' constDef)* ';'
21     ;
22
23 // 基本类型
24 bType
25     : 'int'
26     | 'float'
27     | 'char'
28     ;
29
```

语法规则部分的总体思路是按照对应语法规则对语言的结构进行描述。例：

compUnit 规定一个编译单元是由声明或函数定义完组成（二者间存在至少一个）并由文件结束符 EOF 收尾。

bType 规定了数据的基本类型为 int 整型，float 浮点数或 char。

decl 规定了声明可以是常量声明与变量声明。

```

// 词法规则
✓ Ident
  : [a-zA-Z_] [a-zA-Z0-9_]*
  ;

✓ IntConst
  : [+]? DecimalConst
  | [+]? OctalConst
  | [+]? HexadecConst
  ;

✓ FloatConst
  : ([+]? Digit+ '.' Digit* // 普通形式: 如123.45、123.
  | [+]? '.' Digit+ // 普通形式: 如.45
  | [+]? ( ( Digit+ ('.' Digit*)? | '.' Digit+ ) // 基数部分
  | [Ee] [+]? Digit+ ) // 指数部分
  [fF]?
  ;

// ' '内转义或其他字符 (排除需要转义的字符)
✓ CharConst
  : '\\' (EscapeChar | ~['\\r\n]) '\\'
  ;

```

词法规则部分的总体思路是按照对应的语法规则对语言中的词素进行识别。例：

Ident 用于规定标识符，必须以字母开头，后续可以是大小写字母与阿拉伯数字的排列组合。

```

FloatConst
  : ([+]? Digit+ '.' Digit* // 普通形式: 如123.45、123.
  | [+]? '.' Digit+ // 普通形式: 如.45
  | [+]? ( ( Digit+ ('.' Digit*)? | '.' Digit+ ) // 基数部分
  | [Ee] [+]? Digit+ ) // 指数部分
  [fF]?
  ;

```

这条规则用于识别众多形式的浮点数，例如：1.23e10，.23e10 等。基数部分为可带小数的整数或直接无整数部分的小数。后续跟 e 或者 E 以及整数，结尾可选 f 或者 F 表示 float 类型；

```

// 常量表达式
constExp
  : number
  ;

```

constExp 规则要求使用的必须是常量，因此在这里以 number 写入规则；

```
//非贪婪模式匹配
BlockComment
    : '/*' .*? '*/' -> skip
    ;
```

.表示全字符，'.*'表示若干全字符，后续 '?' 要求前接全字符优先满足“尽量少”规则。

2, main 函数代码:

```
std::ifstream stream(argv[1]);
if (!stream.is_open()) {
    std::cout << "no such file!" << std::endl;
    return 1;
}
```

修改 main.cpp 对应部分原代码为上图，在之后的编译过程中发生如下报错:

```
object of abstract class type "Analysis" is not allowed:C/C++(322)
main.cpp(37, 12): pure virtual function "CACTVisitor::visitDecl" has no override
main.cpp(37, 12): pure virtual function "CACTVisitor::visitConstDecl" has no override
main.cpp(37, 12): pure virtual function "CACTVisitor::visitBType" has no override
main.cpp(37, 12): pure virtual function "CACTVisitor::visitConstDef" has no override
main.cpp(37, 12): pure virtual function "CACTVisitor::visitConstInitVal" has no override
main.cpp(37, 12): pure virtual function "CACTVisitor::visitVarDecl" has no override
main.cpp(37, 12): pure virtual function "CACTVisitor::visitVarDef" has no override
main.cpp(37, 12): pure virtual function "CACTVisitor::visitFuncDef" has no override
```

主要的对部分未被覆写、重写实现的函数进行了实例化，导致发生报错。

```
cd grammar
java -jar ../deps/antlr-4.8-complete.jar -Dlanguage=Cpp \
    -no-listener -visitor ./Hello.g4
```

这里就要考虑到 listener 和 visitor 两种模式之间的区别了:

前者更偏向于被动监听，最终的遍历逻辑实际上是由 ANTLR 来完成的；

而后者则更偏向于主动访问，控制访问流程是要有我们自己来控制的，因此为解决问题，我们需要在上面自己写好各个规则函数的控制流程，如下：

```
using namespace antlr4;

class Analysis : public CACTVisitor {
public:

    // 语法入口规则
    std::any visitCompUnit(CACTParser::CompUnitContext *ctx) override {
        std::cout << "visit CompUnit" << std::endl;
        return visitChildren(ctx);
    }

    // 声明相关规则
    std::any visitDecl(CACTParser::DeclContext *ctx) override {
        std::cout << "visit Decl" << std::endl;
        return visitChildren(ctx);
    }

    std::any visitConstDecl(CACTParser::ConstDeclContext *ctx) override {
        std::cout << "visit ConstDecl" << std::endl;
        return visitChildren(ctx);
    }
}
```

二、问题解答

1、设计编译器的目录结构：

```
cact-compiler/
├── grammar/                # 这里放置写好的语法文件
│   └── CACT.g4
├── src/                    # 主程序及语法树访问器
│   ├── main.cpp           # 编译器主入口，调用 parser 与 visitor
│   └── Analysis.h/.cpp    # 自定义语法树访问器（继承自 CACTVisitor）
├── build/                  # ANTLR 生成的中间文件（Lexer、Parser、Tokens 等）
├── test/                   # 各类测试 .cact 源文件
│   └── 00.test             # 从 00 到 26 各个测试文件所在
```

2、把表达式优先级体现在文法设计中：

主要通过文法的层级嵌套实现，每一层规则可以调用更高优先级的规则使用：

```

// 乘除模表达式
mulExp
: unaryExp
| mulExp ('*' | '/' | '%') unaryExp
;

// 加减表达式
addExp
: mulExp
| addExp ('+' | '-') mulExp
;

```

如上图加减法与乘除法的处理模式，使得乘除法的优先级在加减法之上。

3、设计数值常量的语法规则：

<pre> // 数值 number : IntConst FloatConst CharConst ; </pre>	<pre> IntConst : [+]? DecimalConst [+]? OctalConst [+]? HexadecConst ; FloatConst : ([+]? Digit+ '.' Digit+ // 普通形式，如123.45、123. [+]? '.' Digit+ // 普通形式，如.45 [+]? ((Digit+ ('.' Digit+)? '+' Digit+) // 基数部分 [Ee] [+]? Digit+) // 指数部分 [fF]? ; // '内转义或其他字符（排除需要转义的字符） CharConst : '\'' (EscapeChar ~['\\r\n]) '\'' ; fragment EscapeChar : '\\' [nt\\'"] ; </pre>
---	--

Number 有三种类型分别为：

Int：32 位有符号整数，支持十进制、八进制与十六进制；

Float：遵循 IEEE754，支持普通形式与指数形式；

Char：有符号八位整数，以单引号包围；

4、替换 ANTLR 的默认异常处理规则：

ANTLR 会默认打印错误信息，但不会终止程序，对于报错信息可以自定义一个函数用于错误处理，并在 main.cpp 中替换原错误处理。

```

class MyErrorListener : public BaseErrorListener {
public:
    void syntaxError(Recognizer *recognizer, Token *offendingSymbol,
                    size_t line, size_t charPosInLine,
                    const std::string &msg, std::exception_ptr e) override {
        std::cerr << "Syntax error at line " << line << ":" << charPosInLine
                    << " - " << msg << std::endl;
        exit(1);
    }
};

```

最终打印结果为：

```

[100%] Built target compiler
❯ compiler14@teacher-PowerEdge-M640:~/cact/build$ ./compiler ../test/samples_lex_and_syntax/01_false_hex_num.cact
语法错误: 第 3 行, 第 13 列: extraneous input 'x' expecting {'(', ',', ';' }

```

三、心得体会

通过本次实验，我们初步掌握了如何使用 ANTLR 构建一个简化语言 CACT 的词法分析器和语法分析器，并学会了使用 Visitor 模式遍历语法树。在这个过程中，我理解了语法规则如何体现表达式优先级、如何设计语言的基本结构，以及如何处理语法错误。此外，我也体会到了编译器前端开发的逻辑性和系统性，加深了对编译原理中词法、语法和语义分析过程的理解。这些知识对我后续学习语言实现、中间代码生成等内容非常有帮助。